

LAB SESSION 3

Lab Name: Requirements Engineering & Analysis

Project: MediScript-E — Digital Healthcare Platform

Objectives

- Identify functional and non-functional requirements of MediScript-E
- Apply requirement elicitation techniques
- Prioritize requirements based on system criticality

Theory

Requirements Engineering ensures that the right system is built. For MediScript-E, a digital healthcare platform, accurate requirements are critical because they directly impact patient safety and data security.

Types of Requirements:

- **Functional Requirements (FR):** What the system does — e.g., booking appointments, issuing prescriptions
- **Non-Functional Requirements (NFR):** How the system performs — e.g., response time, security, availability

Poor requirements are the leading cause of project failure. In healthcare systems, incomplete requirements can lead to data breaches or incorrect medical records.

Task 1: Functional Requirement List

FR ID	Functional Requirement	Actor
FR-01	System shall allow users to register as PATIENT, DOCTOR, or ADMIN	All Users
FR-02	System shall send email verification link upon registration	All Users
FR-03	System shall allow verified users to login with email and password	All Users
FR-04	System shall support OAuth login via Google and GitHub	All Users
FR-05	System shall enforce Two-Factor Authentication (2FA) via email OTP when enabled	Patient, Doctor

FR ID	Functional Requirement	Actor
FR-06	System shall allow patients to book appointments with available doctors	Patient
FR-07	System shall allow doctors to confirm, cancel, or complete appointments	Doctor
FR-08	System shall allow doctors to issue digital prescriptions with diagnosis and medications	Doctor
FR-09	System shall allow patients to view and download prescriptions as PDF	Patient
FR-10	System shall allow patients to set medicine reminders with automated email alerts	Patient
FR-11	System shall allow patients to upload and store medical reports (Medical Vault)	Patient
FR-12	System shall allow patients to delete uploaded medical reports	Patient
FR-13	System shall allow users to update profile name and password	All Users
FR-14	System shall allow users to enable or disable 2FA from settings	Patient, Doctor
FR-15	System shall allow admin to view real-time dashboard statistics	Admin
FR-16	System shall allow admin to view and delete users (except self)	Admin
FR-17	System shall allow admin to monitor all appointments with status filters	Admin
FR-18	System shall allow admin to view all contact form submissions	Admin
FR-19	System shall allow unauthenticated users to submit contact form	Public
FR-20	System shall send automated medicine reminder emails via cron job	System

Task 2: Non-Functional Requirement List

NFR ID	Non-Functional Requirement	Category
NFR-01	System shall load pages within 3 seconds under normal load	Performance

NFR ID	Non-Functional Requirement	Category
NFR-02	System shall maintain 99.9% uptime in production	Availability
NFR-03	All passwords shall be hashed using bcryptjs before storage	Security
NFR-04	All database connections shall use SSL/TLS encryption	Security
NFR-05	JWT-based sessions shall expire after 30 days of inactivity	Security
NFR-06	Email verification tokens shall expire within 24 hours	Security
NFR-07	2FA OTP codes shall expire within 10 minutes	Security
NFR-08	System shall support role-based access control (RBAC) for all API routes	Security
NFR-09	Medical file uploads shall be restricted to valid file types and size limits	Security
NFR-10	System shall be fully responsive across mobile, tablet, and desktop	Usability
NFR-11	System UI shall provide clear feedback for all user actions	Usability
NFR-12	System shall use PostgreSQL (Supabase) as production-grade database	Reliability
NFR-13	System codebase shall follow TypeScript strict typing standards	Maintainability
NFR-14	System shall be deployable on Vercel with zero-downtime deployments	Scalability
NFR-15	All API routes shall validate input before processing	Security

Task 3: Priority Matrix

Requirement ID	Description	Priority	Justification
FR-01	User Registration	High	Core entry point of the system
FR-02	Email Verification	High	Prevents fake registrations
FR-03	Credential Login	High	Primary authentication method
FR-04	OAuth Login	Medium	Convenience feature
FR-05	2FA via Email OTP	High	Critical security layer
FR-06	Book Appointment	High	Core patient functionality
FR-07	Manage Appointments	High	Core doctor functionality
FR-08	Issue Prescription	High	Core doctor functionality
FR-09	View/Download Prescription	High	Core patient functionality
FR-10	Medicine Reminders	Medium	Value-added patient feature
FR-11	Medical Vault Upload	Medium	Value-added patient feature
FR-12	Delete Medical Report	Low	Supporting feature
FR-13	Profile/Password Update	Medium	Standard account management

Requirement ID	Description	Priority	Justification
FR-14	2FA Toggle in Settings	Medium	User preference feature
FR-15	Admin Dashboard Stats	High	Critical admin oversight
FR-16	Admin User Management	High	Critical admin control
FR-17	Admin Appointment Monitor	Medium	Admin oversight feature
FR-18	Admin Contact Messages	Low	Supporting admin feature
FR-19	Contact Form	Low	Public communication feature
FR-20	Automated Reminders (Cron)	Medium	Automated system feature
NFR-01	Page Load Performance	High	Directly impacts user experience
NFR-03	Password Hashing	High	Critical security requirement
NFR-04	SSL/TLS Database	High	Critical data protection
NFR-08	RBAC on APIs	High	Prevents unauthorized access
NFR-10	Responsive Design	High	Multi-device accessibility

Key Findings / Learning Outcomes

- Successfully identified **20 Functional Requirements** and **15 Non-Functional Requirements** for MediScript-E
- Learned to categorize requirements by actor (Patient, Doctor, Admin, Public, System)
- Understood that in healthcare systems, security-related NFRs carry equal or higher priority than functional requirements
- Priority matrix helps development team focus on critical features first, ensuring MVP delivery
- Requirement elicitation for MediScript-E involved analyzing user roles, system workflows, and security constraints